```
    @When("I fly from $departure to $destination on $date at $time")
    public void whenIFlyFrom(String departure, String destination,
                             DateTime date, LocalTime time) {
        trip = trips.lookupTrip(departure, destination, time, date);
    }

    @Then("I should earn $points points")
    public void thenIShouldEarn(int expectedPoints) {
        int earnedPoints = member.getPointsFor(trip);
        assertThat(earnedPoints).isEqualTo(expectedPoints);
    }
}
```

Look up the corresponding trip.

Determine how many points are earned for this trip.

Because a new instance of the class containing the step definitions will be created for each scenario, it's safe to use member variables this way. Even if these steps are used in other scenarios, each scenario will have its own member variable.

Although this practice is safe and simple, it can lead to code that's hard to maintain if there are variables used by different scenarios in the same step definition class. This is why many practitioners prefer to put related step definitions (for example, the step definitions for a given feature file or even for a given scenario) in the same class to make the flow of the steps easier to understand.

6.2.5 Using table data from step definitions

In chapter 5 you saw that you can embed tables within the steps to represent more complicated structures. The following scenario uses embedded tabular data to both set up the initial test database and verify the outcomes:

```
Scenario: Transfer points between existing members
    Given the following accounts:
      | owner | points | statusPoints |
      | Jill  | 100,000| 800          |
      | Joe   | 50,000 | 50           |
    When Joe transfers 40000 points to Jill
    Then the accounts should be the following:
      | owner | points | statusPoints |
      | Jill  | 140,000| 800          |
      | Joe   | 10,000 | 50           |
```

Using tabular data like this is an important part of step implementation, and it's something you're likely to do often. In the preceding scenario, for example, you'd use the data in the first step to create these Frequent Flyer accounts, and then read the resulting accounts and compare them with the figures provided in the second table.

Most tools provide APIs to help you do this sort of thing. The capabilities of these APIs vary from one tool to another, and some languages are easier to work with in this regard than others. One very common requirement is to iterate over the table rows and extract the field values, and then use them to set up test data. In JBehave, for example, you could write a step implementation to do this in the following way: